

Task – 1 Process Synchronization and Child Process Monitoring Using waitpid()

```
GNU nano 8.7.1 waitp
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\nChild process started.\n");
        printf("Child PID: %d\n", getpid());
        printf("Child is working...\n");

        sleep(3);

        printf("Child process completed.\n");
        exit(5);
    }
    else
    {
        // Parent process
        printf("\nParent is waiting for child...\n");
        printf("Child PID: %d\n", pid);

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("\nChild terminated normally.\n");
            printf("Child exit status: %d\n",
                WEXITSTATUS(status));
        }
    }

    printf("Parent process completed.\n");
    return 0;
}
```

```
sirichandana@siri:~$ cd ~/practical_memory
sirichandana@siri:~/practical_memory$ nano matpid_demo.c
sirichandana@siri:~/practical_memory$ gcc matpid_demo.c -o matpid_demo
sirichandana@siri:~/practical_memory$ ./matpid_demo
Parent process started.
Parent PID: 616

Parent is waiting for child...
Child PID: 617

Child process started.
Child PID: 617
Child is working...
Child process completed.

Child terminated normally.
Child exit status: 5
Parent process completed.
```

Task – 2 To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

int main(int argc, char *argv[])
{
    char *path;
    char *path_copy;
    char *dir;
    char full_path[1024];
    struct stat file_info;

    if (argc != 2)
    {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    // Retrieve PATH variable
    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH variable:\n%s\n\n", path);

    // Make a copy because strtok modifies the string
    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        perror("strdup");
        return 1;
    }

    // Parse PATH directories
    dir = strtok(path_copy, ":");

    while (dir != NULL)
    {
        snprintf(full_path, sizeof(full_path),
                 "%s/%s", dir, argv[1]);

        // Check whether the file exists
        if (stat(full_path, &file_info) == 0)
        {
            printf("Command found: %s\n", full_path);

            // Check execute permission
            if (access(full_path, X_OK) == 0)
            {
                printf("Executable permission: YES\n");
            }
            else

```

```

        {
            printf("Executable permission: NO\n");
        }

        free(path_copy);
        return 0;
    }

    dir = strtok(NULL, ":");
}

printf("Command '%s' not found in PATH.\n", argv[1]);

free(path_copy);

return 1;
}
|

```

```
sirichandana@siri:~$ cd ~/practical_memory
sirichandana@siri:~/practical_memory$ nano matpid_demo.c
sirichandana@siri:~/practical_memory$ gcc matpid_demo.c -o matpid_demo
sirichandana@siri:~/practical_memory$ ./matpid_demo
Parent process started.
Parent PID: 616

Parent is waiting for child...
Child PID: 617

Child process started.
Child PID: 617
Child is working...
Child process completed.

Child terminated normally.
Child exit status: 5
Parent process completed.
```